

12550200 鄭武謙

Problem 1

From the several views of point below, we can find that the PG.py implement (P1) expression, which called total reward.

First, the policy gradient update rule in the update function follows the classic Monte Carlo method: $G_t = V_t + \gamma V_{t+1}$

This return is then used to update policy parameters.

Secondly, the absence of baseline in the update equation indicates that it is the simplest form of policy gradient. $\theta \leftarrow \theta + \alpha G_t \nabla \log \pi_\theta(a_t | s_t)$

Since there is no subtraction of a baseline or an advantage function, this implementation does not match the structure of (P2) or (P3).

In conclusion, the provided implementation aligns with (P1) - the basic reinforce method without a baseline.

Problem 2

(a)

The implementation uses a linear preference function combined with a Softmax policy

```
prefs = np.dot(state, self.theta)
exp_prefs = np.exp(prefs - np.max(prefs)) # Numerical stability
return exp_prefs / np.sum(exp_prefs)
```

(b)

The code snippet:

```
G = 0
for t in reversed(range(len(states))):
    G = rewards[t] + (0.99 * G) # Discounted return
```

iterates backward over the episode to compute the return G_t , which is defined as: $G_t = V_t + \gamma G_{t+1}$

The reason for using `reversed(range(len(state))` is to efficiently compute the discounted returns in a single pass. Instead of first storing all rewards and computing returns in a separate loop, this method allows us to:

1. start from the last timestep and accumulate returns backward.
2. Efficiently maintain a running sum of G_t
3. Avoid unnecessary storage of intermediate value.

Thus, using `reverse` is an optimization technique that simplifies the implementation.

(C)
This line calculates the gradient of the log policy function:

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \chi(s) [I(a) - \pi_{\theta}(\cdot|s)]$$

where:

$\chi(s)$ is the input vector.

$I(a)$ is an indicator function (1 if action was taken, 0 otherwise)

$\pi_{\theta}(\cdot|s)$ is the policy probability vector over all actions

The implementation breaks this into two components:

1. For the selected action a_t , the gradient is:

$$\nabla_{\theta} \log \pi_{\theta}(a_t|s) = \chi(s) (1 - \pi_{\theta}(a_t|s))$$

2. For all other actions, the gradient is negative weighted by the probability:

$$\nabla_{\theta} \log \pi_{\theta}(a'|s) = -\chi(s) \pi_{\theta}(a'|s) \quad (\text{for } a' \neq a_t)$$

Problem 3

while the implementation follows the fundamental reinforce algorithm, there are some potential improvements:

Missing Baseline $\rightarrow$ High Variance Issue

Issue:

The current implementation does not use a baseline, which can lead to high variance in policy updates.

Solution:

Instead of directly using G_t , subtracting a baseline (e.g., estimated state value function $V(s)$) reduces variance:

$$Q \leftarrow Q + \alpha (G_t - V(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

This can be implemented by estimating $V(s)$ via a separate neural network.